Almacenamiento y Procesamiento de Grandes Datos

Subtaller 3: Apache Spark — Arquitectura Medallion (Colab)

Objetivo

Aprender los fundamentos de Apache Spark mediante la implementación de una arquitectura **Medallion** (Bronze → Silver → Gold) usando un dataset de ventas de e-commerce.

Arquitectura Medallion

```
JSON crudo                  Bronze               Silver                  Gold
(data/)          →   (datos sin limpiar)  →  (datos limpios)  →  (datos agregados)
ventas_ecommerce.json   Parquet raw         Parquet limpio         Parquet analítico
```

Contenido

1. Instalación y verificación de PySpark
2. Conceptos clave de Spark
3. Crear SparkSession
4. Capa Bronze — Ingestar datos crudos
5. Capa Silver — Limpiar y transformar
6. Capa Gold — Agregar para análisis
7. Consultas analíticas sobre Gold

---

1. Instalación y verificación

```
# Instalación automática para Colab (solo si hace falta)
import importlib.util
import subprocess
import sys

if importlib.util.find_spec('pyspark') is None:
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', '-q', 'pyspark'])

import pyspark
print(f'PySpark version: {pyspark.__version__}')
```

```
PySpark version: 4.0.2
```

---

2. Conceptos clave de Spark

Concepto	Descripción
SparkSession	Punto de entrada a Spark. Una por aplicación.
DataFrame	Tabla distribuida con columnas tipadas. Equivale a un DataFrame de Pandas pero distribuido.
RDD	Resilient Distributed Dataset — capa de bajo nivel (hoy se prefiere DataFrame).
Transformación	Operación lazy: `filter`, `select`, `groupBy`. No ejecuta hasta una acción.
Acción	Dispara la ejecución: `show()`, `count()`, `write`, `collect()`.
Partición	Chunk del dataset distribuido entre nodos (en local, entre CPU cores).
Lazy Evaluation	Spark construye un plan de ejecución y lo optimiza antes de correrlo.
Catalyst Optimizer	Motor interno que optimiza el plan de consulta automáticamente.

---

3. Crear SparkSession

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.types import *
import os

# SparkSession es el punto de entrada de cualquier aplicación Spark
spark = SparkSession.builder \
    .appName('TallerMedallion') \
    .master('local[*]') \
    .config('spark.sql.shuffle.partitions', '4') \
    .getOrCreate()

# Reducir logs para mayor legibilidad
spark.sparkContext.setLogLevel('WARN')

print(f'Spark version: {spark.version}')
print(f'Master: {spark.sparkContext.master}')
```

```
Spark version: 4.0.2
Master: local[*]
```

```
# Definir rutas de la arquitectura Medallion
BASE_PATH = './medallion'
BRONZE_PATH = f'{BASE_PATH}/bronze/ventas'
SILVER_PATH = f'{BASE_PATH}/silver/ventas'
GOLD_PATH   = f'{BASE_PATH}/gold'

# Ruta del dataset fuente
SOURCE_JSON = '/datalab/ventas_ecommerce.json'

# Crear directorios locales
for path in [BRONZE_PATH, SILVER_PATH, GOLD_PATH]:
    os.makedirs(path, exist_ok=True)

print('Rutas creadas:')
```

```
for path in [BRONZE_PATH, SILVER_PATH, GOLD_PATH]:
    print(f'  {path}')
```

```
Rutas creadas:
  ./medallion/bronze/ventas
  ./medallion/silver/ventas
  ./medallion/gold
```

4. Capa Bronze — Ingesta de datos crudos

> **Bronze**: datos tal como llegan de la fuente. Sin limpieza. Solo se agrega metadata de ingesta.

```
# Leer el JSON crudo
df_raw = spark.read \
    .option('multiline', 'true') \
    .json(SOURCE_JSON)

print('=== Schema del dataset crudo ===')
df_raw.printSchema()
```

```
=== Schema del dataset crudo ===
root
 |-- category: string (nullable = true)
 |-- city: string (nullable = true)
 |-- country: string (nullable = true)
 |-- customer_email: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- discount: double (nullable = true)
 |-- order_date: string (nullable = true)
 |-- order_id: string (nullable = true)
 |-- product_id: string (nullable = true)
 |-- product_name: string (nullable = true)
 |-- quantity: long (nullable = true)
 |-- status: string (nullable = true)
 |-- unit_price: double (nullable = true)
```

```
# Explorar los datos
print(f'Total registros: {df_raw.count()}')
df_raw.show(5, truncate=False)
```

```
Total registros: 30
+-----------+------------+--------+-------------------------+-----------+---------------
|category   |city        |country |customer_email           |customer_id|customer_name
+-----------+------------+--------+-------------------------+-----------+---------------
|Electrónica|Bogotá      |Colombia|ana.garcia@email.com     |C001       |Ana García
|Electrónica|Medellín    |Colombia|carlos.lopez@email.com   |C002       |Carlos López
|Muebles    |Cali        |Colombia|maria.rodriguez@email.com|C003       |María Rodrígue
|Electrónica|Bogotá      |Colombia|ana.garcia@email.com     |C001       |Ana García
|Libros     |Barranquilla|Colombia|pedro.martinez@email.com |C004       |Pedro Martínez
+-----------+------------+--------+-------------------------+-----------+---------------
```

```
only showing top 5 rows
```

```
# Bronze: agregar metadata de ingesta y guardar en Parquet
from datetime import datetime

df_bronze = df_raw \
    .withColumn('ingestion_timestamp', F.current_timestamp()) \
    .withColumn('source_file', F.lit(SOURCE_JSON)) \
    .withColumn('ingestion_date', F.current_date())

# Escribir en formato Parquet (columnar, eficiente para analytics)
df_bronze.write \
    .mode('overwrite') \
    .partitionBy('ingestion_date') \
    .parquet(BRONZE_PATH)

print(f'Bronze guardado en: {BRONZE_PATH}')
print(f'Registros en Bronze: {df_bronze.count()}')
```

```
Bronze guardado en: ./medallion/bronze/ventas
Registros en Bronze: 30
```

```
# Verificar lo que se escribió en Bronze
df_bronze_check = spark.read.parquet(BRONZE_PATH)
df_bronze_check.select(
    'order_id', 'customer_name', 'product_name',
    'unit_price', 'status', 'ingestion_timestamp'
).show(5, truncate=False)
```

```
+--------+---------------+-------------------------+----------+---------+---------------
|order_id|customer_name  |product_name             |unit_price|status   |ingestion_times
+--------+---------------+-------------------------+----------+---------+---------------
|ORD-001 |Ana García     |Laptop HP 15             |1200.0    |entregado|2026-04-03 18:5
|ORD-002 |Carlos López   |Auriculares Sony WH      |150.0     |entregado|2026-04-03 18:5
|ORD-003 |María Rodríguez|Silla Ergonómica         |350.0     |enviado  |2026-04-03 18:5
|ORD-004 |Ana García     |Mouse Inalámbrico        |45.0      |entregado|2026-04-03 18:5
|ORD-005 |Pedro Martínez |Libro Python Data Science|35.0      |entregado|2026-04-03 18:5
+--------+---------------+-------------------------+----------+---------+---------------
only showing top 5 rows
```

---

5. Capa Silver — Limpieza y transformaciones

> **Silver**: datos limpios, validados y enriquecidos. Se corrigen tipos, se eliminan duplicados y se calculan columnas derivadas.

```
# Leer desde Bronze
df_b = spark.read.parquet(BRONZE_PATH)

# 1. Verificar nulos
print('=== Conteo de nulos por columna ===')
```

```
df_b.select([
    F.count(F.when(F.col(c).isNull(), c)).alias(c)
    for c in ['order_id', 'customer_id', 'product_id', 'quantity', 'unit_price', 'stat
]).show()
```

```
=== Conteo de nulos por columna ===
+--------+-----------+----------+--------+----------+------+
|order_id|customer_id|product_id|quantity|unit_price|status|
+--------+-----------+----------+--------+----------+------+
|       0|          0|         0|       0|         0|     0|
+--------+-----------+----------+--------+----------+------+
```

```
# 2. Verificar duplicados
total = df_b.count()
distinct = df_b.dropDuplicates(['order_id']).count()
print(f'Total registros: {total}')
print(f'Registros únicos por order_id: {distinct}')
print(f'Duplicados: {total - distinct}')
```

```
Total registros: 30
Registros únicos por order_id: 30
Duplicados: 0
```

```
# 3. Verificar valores de la columna status
df_b.groupBy('status').count().orderBy('count', ascending=False).show()
```

```
+---------+-----+
|   status|count|
+---------+-----+
|entregado|   23|
|  enviado|    4|
|cancelado|    3|
+---------+-----+
```

```
# Transformaciones Silver
df_silver = df_b \
    .dropDuplicates(['order_id']) \
    .filter(F.col('order_id').isNotNull()) \
    .filter(F.col('customer_id').isNotNull()) \
    .withColumn('order_date', F.to_date('order_date', 'yyyy-MM-dd')) \
    .withColumn('order_year',  F.year('order_date')) \
    .withColumn('order_month', F.month('order_date')) \
    .withColumn('unit_price',  F.col('unit_price').cast(DoubleType())) \
    .withColumn('quantity',    F.col('quantity').cast(IntegerType())) \
    .withColumn('discount',    F.col('discount').cast(DoubleType())) \
    .withColumn('gross_amount', F.round(F.col('quantity') * F.col('unit_price'), 2)) \
    .withColumn('net_amount',   F.round(
        F.col('quantity') * F.col('unit_price') * (1 - F.col('discount')), 2
    )) \
    .withColumn('is_cancelled', F.when(F.col('status') == 'cancelado', True).otherwise
    .withColumn('status', F.lower(F.trim('status'))) \
    .drop('ingestion_date', 'source_file')
```

```
print(f'Silver: {df_silver.count()} registros')
df_silver.printSchema()
```

```
Silver: 30 registros
root
 |-- category: string (nullable = true)
 |-- city: string (nullable = true)
 |-- country: string (nullable = true)
 |-- customer_email: string (nullable = true)
 |-- customer_id: string (nullable = true)
 |-- customer_name: string (nullable = true)
 |-- discount: double (nullable = true)
 |-- order_date: date (nullable = true)
 |-- order_id: string (nullable = true)
 |-- product_id: string (nullable = true)
 |-- product_name: string (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- status: string (nullable = true)
 |-- unit_price: double (nullable = true)
 |-- ingestion_timestamp: timestamp (nullable = true)
 |-- order_year: integer (nullable = true)
 |-- order_month: integer (nullable = true)
 |-- gross_amount: double (nullable = true)
 |-- net_amount: double (nullable = true)
 |-- is_cancelled: boolean (nullable = false)
```

```
# Guardar Silver particionado por año y mes
df_silver.write \
    .mode('overwrite') \
    .partitionBy('order_year', 'order_month') \
    .parquet(SILVER_PATH)

print(f'Silver guardado en: {SILVER_PATH}')
```

```
Silver guardado en: ./medallion/silver/ventas
```

```
# Vista previa de Silver
spark.read.parquet(SILVER_PATH).select(
    'order_id', 'customer_name', 'product_name', 'category',
    'gross_amount', 'net_amount', 'status', 'order_date'
).show(8, truncate=False)
```

```
+--------+---------------+-------------------------+-----------+------------+-----------
|order_id|customer_name  |product_name             |category   |gross_amount|net_amount
+--------+---------------+-------------------------+-----------+------------+-----------
|ORD-013 |María Rodríguez|Teclado Mecánico         |Electrónica|120.0       |108.0
|ORD-014 |Isabela Moreno |Escritorio de Pie        |Muebles    |500.0       |475.0
|ORD-015 |Felipe Vargas  |Auriculares Sony WH      |Electrónica|150.0       |150.0
|ORD-016 |Natalia Ríos   |Laptop HP 15             |Electrónica|1200.0      |1020.0
|ORD-017 |Pedro Martínez |Cámara Web HD            |Electrónica|80.0        |80.0
|ORD-018 |Camilo Ospina  |Monitor 24 pulgadas      |Electrónica|560.0       |504.0
|ORD-019 |Laura Sánchez  |Silla Ergonómica         |Muebles    |350.0       |350.0
|ORD-020 |Daniela Castro |Libro Python Data Science|Libros     |70.0        |63.0
+--------+---------------+-------------------------+-----------+------------+-----------
only showing top 8 rows
```

6. Capa Gold — Agregaciones para análisis

> **Gold**: datos listos para consumo analítico. Se crean tablas de hechos y dimensiones, KPIs y reportes.

```
df_s = spark.read.parquet(SILVER_PATH)
df_active = df_s.filter(F.col('status') != 'cancelado')

# --- Gold 1: KPIs por mes ---
gold_monthly = df_active \
    .groupBy('order_year', 'order_month') \
    .agg(
        F.count('order_id').alias('total_orders'),
        F.countDistinct('customer_id').alias('unique_customers'),
        F.round(F.sum('gross_amount'), 2).alias('gross_revenue'),
        F.round(F.sum('net_amount'), 2).alias('net_revenue'),
        F.round(F.avg('net_amount'), 2).alias('avg_order_value')
    ) \
    .orderBy('order_year', 'order_month')

gold_monthly.write.mode('overwrite').parquet(f'{GOLD_PATH}/kpis_mensuales')
print('Gold: KPIs mensuales')
gold_monthly.show()
```

```
Gold: KPIs mensuales
+----------+-----------+------------+----------------+-------------+-----------+--------
|order_year|order_month|total_orders|unique_customers|gross_revenue|net_revenue|avg_orde
+----------+-----------+------------+----------------+-------------+-----------+--------
|      2024|          1|          11|               9|       3430.0|    3198.75|
|      2024|          2|          11|              11|       4860.0|     4454.0|
|      2024|          3|           5|               5|       1450.0|     1329.5|
+----------+-----------+------------+----------------+-------------+-----------+--------
```

```
# --- Gold 2: Ventas por categoría de producto ---
gold_category = df_active \
    .groupBy('category') \
    .agg(
        F.count('order_id').alias('num_orders'),
        F.sum('quantity').alias('units_sold'),
        F.round(F.sum('net_amount'), 2).alias('net_revenue'),
        F.round(F.avg('unit_price'), 2).alias('avg_price')
    ) \
    .orderBy('net_revenue', ascending=False)

gold_category.write.mode('overwrite').parquet(f'{GOLD_PATH}/ventas_por_categoria')
print('Gold: Ventas por categoría')
gold_category.show()
```

```
Gold: Ventas por categoría
+-----------+----------+----------+-----------+---------+
```

```
|   category|num_orders|units_sold|net_revenue|avg_price|
+-----------+----------+----------+-----------+---------+
|Electrónica|        17|        25|     5920.5|   326.18|
|    Muebles|         6|         7|     2762.5|    425.0|
|     Libros|         4|        10|     299.25|     35.0|
+-----------+----------+----------+-----------+---------+
```

```
# --- Gold 3: Top clientes por revenue ---
gold_customers = df_active \
    .groupBy('customer_id', 'customer_name', 'city') \
    .agg(
        F.count('order_id').alias('num_orders'),
        F.round(F.sum('net_amount'), 2).alias('total_spent'),
        F.round(F.avg('net_amount'), 2).alias('avg_order')
    ) \
    .orderBy('total_spent', ascending=False)

gold_customers.write.mode('overwrite').parquet(f'{GOLD_PATH}/top_clientes')
print('Gold: Top clientes')
gold_customers.show(10)
```

```
Gold: Top clientes
+-----------+---------------+---------+----------+-----------+---------+
|customer_id|  customer_name|     city|num_orders|total_spent|avg_order|
+-----------+---------------+---------+----------+-----------+---------+
|       C001|     Ana García|   Bogotá|         3|     1405.0|   468.33|
|       C007|  Sofía Herrera| Medellín|         2|     1232.0|    616.0|
|       C013|   Natalia Ríos|   Bogotá|         1|     1020.0|   1020.0|
|       C008|   Diego Flores|     Cali|         2|      742.0|    371.0|
|       C006|    Juan Torres|   Bogotá|         2|      620.0|    310.0|
|       C010| Andrés Jiménez|   Bogotá|         2|      610.0|    305.0|
|       C002|   Carlos López| Medellín|         2|      580.0|    290.0|
|       C014|  Camilo Ospina|Manizales|         1|      504.0|    504.0|
|       C011| Isabela Moreno|   Bogotá|         1|      475.0|    475.0|
|       C003|María Rodríguez|     Cali|         2|      440.5|   220.25|
+-----------+---------------+---------+----------+-----------+---------+
only showing top 10 rows
```

```
# --- Gold 4: Top productos ---
gold_products = df_active \
    .groupBy('product_id', 'product_name', 'category') \
    .agg(
        F.sum('quantity').alias('units_sold'),
        F.round(F.sum('net_amount'), 2).alias('net_revenue')
    ) \
    .orderBy('net_revenue', ascending=False)

gold_products.write.mode('overwrite').parquet(f'{GOLD_PATH}/top_productos')
print('Gold: Top productos')
gold_products.show()
```

```
Gold: Top productos
+----------+--------------------+-----------+----------+-----------+
|product_id|        product_name|   category|units_sold|net_revenue|
+----------+--------------------+-----------+----------+-----------+
|      P101|        Laptop HP 15|Electrónica|         3|     3180.0|
```

```
|      P901|   Escritorio de Pie|    Muebles|         3|     1450.0|
|      P310|    Silla Ergonómica|    Muebles|         4|     1312.5|
|      P801| Monitor 24 pulgadas|Electrónica|         4|     1064.0|
|      P205| Auriculares Sony WH|Electrónica|         6|      832.5|
|      P602|    Teclado Mecánico|Electrónica|         3|      342.0|
|      P501|Libro Python Data...|     Libros|        10|     299.25|
|      P410|   Mouse Inalámbrico|Electrónica|         6|      270.0|
|      P710|       Cámara Web HD|Electrónica|         3|      232.0|
+----------+--------------------+-----------+----------+-----------+
```

---

7. Consultas analíticas sobre Gold

```
# Registrar Silver como vista temporal para usar Spark SQL
df_s.createOrReplaceTempView('ventas')

# Spark SQL — exactamente igual a SQL estándar
spark.sql('''
SELECT
    category,
    COUNT(order_id)       AS num_ordenes,
    SUM(quantity)         AS unidades,
    ROUND(SUM(net_amount),2) AS ingresos_netos
FROM ventas
WHERE status != 'cancelado'
GROUP BY category
ORDER BY ingresos_netos DESC
''').show()
```

```
+-----------+-----------+--------+--------------+
|   category|num_ordenes|unidades|ingresos_netos|
+-----------+-----------+--------+--------------+
|Electrónica|         17|      25|        5920.5|
|    Muebles|          6|       7|        2762.5|
|     Libros|          4|      10|        299.25|
+-----------+-----------+--------+--------------+
```

```
# Tasa de cancelación
spark.sql('''
SELECT
    CONCAT(order_year, '-', LPAD(order_month, 2, '0')) AS mes,
    COUNT(*) AS total,
    SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) AS cancelados,
    ROUND(100.0 * SUM(CASE WHEN status = 'cancelado' THEN 1 ELSE 0 END) / COUNT(*), 1)
FROM ventas
GROUP BY mes
ORDER BY mes
''').show()
```

```
+-------+-----+----------+---------------+
|    mes|total|cancelados|pct_cancelacion|
+-------+-----+----------+---------------+
|2024-01|   12|         1|            8.3|
```

```
|2024-02|   12|         1|            8.3|
|2024-03|    6|         1|           16.7|
+-------+-----+----------+---------------+
```

```
# Window function — ranking de clientes por revenue dentro de cada ciudad
from pyspark.sql.window import Window

window_city = Window.partitionBy('city').orderBy(F.desc('total_spent'))

spark.read.parquet(f'{GOLD_PATH}/top_clientes') \
    .withColumn('rank_in_city', F.rank().over(window_city)) \
    .filter(F.col('rank_in_city') == 1) \
    .select('city', 'customer_name', 'total_spent', 'num_orders') \
    .orderBy('total_spent', ascending=False) \
    .show()
```

```
+------------+---------------+-----------+----------+
|        city|  customer_name|total_spent|num_orders|
+------------+---------------+-----------+----------+
|      Bogotá|     Ana García|     1405.0|         3|
|    Medellín|  Sofía Herrera|     1232.0|         2|
|        Cali|   Diego Flores|      742.0|         2|
|   Manizales|  Camilo Ospina|      504.0|         1|
|      Cúcuta|  Mauricio Peña|      382.5|         1|
|   Cartagena|  Laura Sánchez|      350.0|         1|
|     Pereira|  Felipe Vargas|      150.0|         1|
| Santa Marta|Sebastián Muñoz|       90.0|         1|
|Barranquilla| Pedro Martínez|      89.25|         1|
| Bucaramanga| Valentina Cruz|       35.0|         1|
+------------+---------------+-----------+----------+
```

```
# Cerrar SparkSession
spark.stop()
print('SparkSession cerrada.')
```

```
SparkSession cerrada.
```